

Raw Python vs. LangChain — Write-Up

Week 2, Day 2 — Tools, Chains, Memory & Your First Framework Agent

Raw-Python vs. LangChain Comparison

The four core pieces built by hand on Day 1 map directly onto four LangChain classes: the raw API client → **ChatGoogleGenerativeAI**; the hand-written tool schema + function → the **@tool** decorator (which reads the function's docstring as its description, exactly like Day 1's manual JSON schema description field); the hand-written while-loop → **create_tool_calling_agent** + **AgentExecutor**; and the manually-resent message list → LangChain's memory utilities. Nothing here is a new concept — LangChain gives each piece a reusable, named class instead of project-specific code.

LCEL's pipe (|) operator is Python's overloaded `__or__`: every component (prompt, LLM, output parser) implements the same `Runnable` interface, and | chains two `Runnable`s into a `RunnableSequence` whose `.invoke()` feeds each component's output into the next, left to right.

Annotated Reasoning Trace (Task 3, real run)

Query: "What is the price of the iPhone 15, and what would that price be with a 10% discount?" — genuinely required two tool calls, run against live Gemini via `AgentExecutor(verbose=True)`.

[ACT]

Invoking: `get_product_price` with `{'product_name': 'iPhone 15'}`

[OBSERVE]

`{"product": "iPhone 15", "price_usd": 799, "category": "smartphone", "brand": "Apple"}`

[ACT]

Invoking: `calculator` with `{'expression': '799 * 0.9'}`

[OBSERVE]

719.1

[REASON+FINAL]

The original price of the iPhone 15 is \$799.00. With a 10% discount, the price would be \$719.10.

Similar to Day 1: the underlying loop is identical — reason, act, observe, repeat until a final answer; the information carried (tool name, arguments, return value) is the same information Day 1's [REASON]/[ACT]/[OBSERVE] prints showed. **What's now hidden:** the actual loop code (the while that checks for tool calls and re-invokes the model) lives inside `AgentExecutor` — it is no longer written or visible. The exact prompt LangChain assembles around `agent_scratchpad` is also invisible unless deliberately inspected.

Multi-Turn Memory (Task 4, real run)

"Find the price of the iPhone 15" → "Now compare it to the Samsung Galaxy S24" → "Which one should I recommend to a budget-conscious client?" — each turn correctly used prior context (comparing the two prices from memory, then reasoning to a recommendation) without being told the prices again, confirming multi-turn context handling worked end to end.

Failure Modes & Abstraction Leakiness Observed

The first two issues below were hit live during development and had to be fixed before this run; the output-shape issue is visible directly in the trace above and in the tool-error demo below.

#	What happened	Fix / mitigation
1	<code>RunnableWithMessageHistory</code> (the textbook memory tool) crashed on turn 2: it stores the model's raw output message, and Gemini's "thinking" signature metadata blocks don't survive being replayed back through LangChain's message-coercion layer.	Managed memory manually instead — same idea <code>ConversationBufferMemory</code> is built on (a plain list), re-wrapping each final answer as a clean <code>AIMessage</code> .
2	<code>AgentExecutor(handle_tool_error=True)</code> alone did NOT catch a <code>ToolException</code> raised inside a tool — the executor-level flag governs output-parsing errors, not tool execution errors.	Had to also set <code>handle_tool_error=True</code> on each individual Tool object (a second, undocumented-feeling step).
3	The agent's final "output" is not a clean string — it is a raw list of Gemini content blocks (e.g. <code>[{'type': 'text', 'text': ..., 'extras': {'signature': ...}}]</code>), visible in every FINAL ANSWER print in this run, including the tool-error demo below.	Requires manually extracting <code>.text</code> from the block before treating the output as a plain string — not handled automatically.

Tool-Error Recovery, Real Run

Query: "What is the price of the OnePlus 12?" (not in the product database). With both fixes applied (per-tool `handle_tool_error=True`), the raised `ToolException` was caught cleanly — no crash — and the model responded: "I'm sorry, but the OnePlus 12 is not in the product database. The available products are: iPhone 15, Samsung Galaxy S24, Google Pixel 8."

What LangChain Made Easier vs. What Leaked

LangChain genuinely simplified **structured output** (`with_structured_output(...)` coerced the final answer into a typed Pydantic object in one line, verified: Samsung Galaxy S24, \$699, with a generated recommendation and reasoning) and the **shape** of assembling an agent (model + tools + prompt wired together without hand-writing the loop). But every failure mode above was a real, reproduced abstraction leak, not a hypothetical one: memory replay broke on the underlying model's own message format, tool-error handling needed configuration in two separate places to actually work, and the final output type wasn't guaranteed to be the clean string the API surface implies. Day 1's raw approach never hit any of these, precisely because every piece of state was explicit, hand-written Python instead of internals hidden two layers deep inside a framework.